

BÁO CÁO PHÂN TÍCH TOÁN HỌC, PHÂN CHIA CÔNG VIỆC VÀ HƯỚNG DẪN TRIỂN KHAI CHI TIẾT

Dự án: Hệ thống trực quan hóa cấu trúc rời rạc & giải thuật đồ thị

TÓM TẮT DỰ ÁN (ABSTRACT):

Tài liệu này đóng vai trò là hướng dẫn kỹ thuật và kế hoạch phân công công việc chi tiết dành cho nhóm phát triển gồm 5 thành viên để xây dựng ứng dụng trực quan hóa 9 thuật toán đồ thị cơ bản và nâng cao của môn học Cấu trúc rời rạc. Điểm đặc biệt của dự án này là sự phân chia công bằng, trong đó cả 5 thành viên đều trực tiếp lập trình cả phần thuật toán thuần (Backend), thuật toán hoạt họa (Generator) và lập trình điều khiển giao diện (GUI) liên quan đến phần việc của mình. Tài liệu bao gồm cơ sở toán học chi tiết, mã nguồn khung định hướng (Skeleton Code) cho từng vai trò, tiến độ thực hiện và các tiêu chuẩn chất lượng (DoD) để đảm bảo mã nguồn đồng nhất, sạch sẽ.

MỤC LỤC TÀI LIỆU:

- Tổng quan kiến trúc hệ thống (Trang 2)
- Phân tích toán học & Thuật toán chi tiết (Trang 2-4)
- Bảng phân chia công việc nhóm (Trang 5)
- Chi tiết phần việc và Skeleton Code cho từng thành viên (Trang 5-9)
- Quy trình phối hợp và thứ tự triển khai (Trang 10)
- Tiêu chuẩn hoàn thành và Clean Code (Trang 10)

1. TỔNG QUAN KIẾN TRÚC HỆ THỐNG

Hệ thống áp dụng mô hình phân tách mối quan tâm (Separation of Concerns) nhằm đảm bảo code thuật toán độc lập hoàn toàn với phần giao diện hiển thị, tạo điều kiện thuận lợi cho việc tái sử dụng mã nguồn:

- Lớp Dữ liệu (Utils): Thực hiện chuyển đổi qua lại giữa các dạng biểu diễn đồ thị (Adjacency Matrix, Adjacency List, Edge List) cho cả đồ thị có hướng và vô hướng.
- Lớp Thuật toán (Backend): Bao gồm 2 phiên bản độc lập. Phiên bản thuần (nằm trong thư mục algorithms/pure/) nhận đầu vào và trả về kết quả cuối cùng theo cách tuyến tính. Phiên bản hoạt họa (nằm trong algorithms/) được triển khai bằng Generator (từ khóa yield) để trả về từng bước chạy cho giao diện hiển thị.
- Lớp Giao diện (GUI): Viết bằng tkinter, nhận sự kiện từ chuột (vẽ đỉnh, vẽ cạnh) và gọi vòng lặp thời gian để liên tục lấy trạng thái từ generator để cập nhật giao diện canvas.

2. PHÂN TÍCH TOÁN HỌC & GIẢI THUẬT CHI TIẾT

2.1. Duyệt đồ thị (BFS & DFS)

Duyệt đồ thị là quá trình đi qua tất cả các đỉnh của đồ thị một cách hệ thống:

- BFS (Breadth-First Search - Duyệt rộng): Sử dụng cấu trúc dữ liệu Hàng đợi Queue (FIFO). Xuất phát từ đỉnh nguồn \$\$\$, loang đều ra xung quanh để duyệt các đỉnh có khoảng cách \$\$\$ trước khi chuyển sang khoảng cách \$\$\$+1. Thích hợp để tìm đường đi ngắn nhất trên đồ thị không trọng số. Độ phức tạp: \$\$\$O(V+E)\$\$\$ thời gian, \$\$\$O(V)\$\$\$ không gian.
- DFS (Depth-First Search - Duyệt sâu): Sử dụng cấu trúc dữ liệu Ngăn xếp Stack (LIFO) (hoặc đệ quy hệ thống). Thuật toán đi sâu tối đa theo từng nhánh cho tới khi không đi được nữa thì thực hiện quay lui (Backtracking). Độ phức tạp: \$\$\$O(V+E)\$\$\$ thời gian, \$\$\$O(V)\$\$\$ không gian.

2.2. Kiểm tra đồ thị hai phía (Bipartite Graph Check)

Định nghĩa toán học: Một đồ thị vô hướng $G = (V, E)$ là đồ thị hai phía nếu tồn tại phân hoạch $V = V_1 \cup V_2$ với $V_1 \cap V_2 = \emptyset$ sao cho mọi cạnh $(u, v) \in E$ đều kết nối một đỉnh thuộc V_1 với một đỉnh thuộc V_2 .

Giải thuật: Sử dụng thuật toán tô màu đồ thị bằng 2 màu (0 và 1). Khởi tạo tất cả đỉnh chưa có màu. Chọn một đỉnh tô màu 0. Sử dụng DFS hoặc BFS loang ra các đỉnh kề: nếu đỉnh kề chưa tô màu, tô màu ngược lại; nếu đỉnh kề đã tô màu và trùng màu với đỉnh đang xét, kết luận đồ thị chứa chu trình lẻ (không là đồ thị hai phía) và truy vết ngược qua mảng cha parent để xuất chu trình lẻ làm minh chứng.

2.3. Tìm đường đi ngắn nhất (Dijkstra)

Định nghĩa bài toán: Cho đồ thị có trọng số không âm $G = (V, E, w)$ với $w(u, v) \geq 0$, tìm đường đi ngắn nhất từ đỉnh nguồn s tới tất cả các đỉnh khác.

Thuật toán: Giải thuật tham lam (Greedy). Duy trì mảng khoảng cách tạm thời $d[v]$ từ s . Ở mỗi bước lặp, chọn đỉnh u chưa cố định có khoảng cách $d[u]$ nhỏ nhất, cố định khoảng cách này. Thực hiện nới lỏng (Relaxation) cho tất cả các đỉnh kề v chưa cố định của u : nếu $d[u] + w(u, v) < d[v]$, cập nhật $d[v] = d[u] + w(u, v)$ và ghi nhận $prev[v] = u$.

Độ phức tạp: Sử dụng Hàng đợi ưu tiên (Binary Min-Heap) giúp thuật toán đạt hiệu năng $O(E \log V)$ thời gian và $O(V)$ không gian.

2.4. Cây khung nhỏ nhất MST (Kruskal & Prim)

Định nghĩa bài toán: Cho đồ thị vô hướng liên thông có trọng số $G = (V, E, w)$. Tìm cây con chứa tất cả đỉnh của đồ thị có đúng $|V|-1$ cạnh sao cho tổng trọng số các cạnh là nhỏ nhất.

- Thuật toán Kruskal (Duyệt theo cạnh): Sắp xếp danh sách cạnh theo trọng số tăng dần. Duyệt qua từng cạnh: nếu hai đỉnh của cạnh nằm ở hai thành phần liên thông khác nhau, kết nạp cạnh đó vào MST. Sử dụng cấu trúc dữ liệu Disjoint Set Union (DSU) với kỹ thuật nén đường đi (Path Compression) để kiểm tra chu trình và hợp nhất tập hợp với độ phức tạp gần như $O(1)$. Tổng độ phức tạp: $O(E \log E)$ thời gian (do chi phí sắp xếp cạnh).

- Thuật toán Prim (Duyệt theo đỉnh): Bắt đầu từ một đỉnh gốc, xây dựng cây khung bằng cách liên tục kết nạp cạnh có trọng số nhỏ nhất nối từ một đỉnh đã nằm trong cây ra một đỉnh chưa nằm trong cây. Sử dụng Min-Heap để quản lý các cạnh ứng viên. Tổng độ phức tạp: $O(E \log V)$ thời gian.

2.5. Luồng cực đại (Ford-Fulkerson / Edmonds-Karp)

Bài toán mạng luồng: Cho mạng luồng $G = (V, E)$ với dung lượng cạnh $c(u, v) \geq 0$, đỉnh nguồn s , và đỉnh đích t . Tìm luồng có giá trị lớn nhất từ s đến t thỏa mãn ràng buộc dung lượng ($0 \leq f(u, v) \leq c(u, v)$) và bảo toàn luồng tại các đỉnh trung gian.

Thuật toán Edmonds-Karp: Là một cải tiến của Ford-Fulkerson. Thuật toán liên tục tìm đường tăng luồng ngắn nhất (ít cạnh nhất) bằng cách duyệt **BFS** trên đồ thị dư G_f . Tìm dung lượng dư nhỏ nhất trên đường đi này ($\Delta f = \text{ext}\{\text{bottleneck}\}$). Tăng luồng thực tế dọc theo đường tăng luồng thêm Δf (tăng luồng cạnh xuôi, giảm luồng cạnh ngược trên đồ thị dư). Thuật toán dừng lại khi không còn đường đi từ s đến t trên đồ thị dư.

Độ phức tạp: Đảm bảo số lần lặp tối đa và kết thúc trong $O(V E^2)$ thời gian.

2.6. Chu trình & Đường đi Euler (Fleury & Hierholzer)

Định lý tồn tại Euler: Đồ thị vô hướng liên thông có chu trình Euler khi tất cả các đỉnh đều có bậc chẵn; có đường đi Euler khi có đúng 0 hoặc 2 đỉnh bậc lẻ.

- Thuật toán Fleury: Đi tham lam từ đỉnh xuất phát. Ở mỗi bước chọn cạnh đi tiếp chưa viếng thăm sao cho tránh đi qua cạnh cầu (bridge) của đồ thị còn lại, trừ khi không còn lựa chọn nào khác. Độ phức tạp thời gian: $O(E^2)$ do mỗi bước phải kiểm tra cầu bằng cách duyệt đồ thị phụ.
- Thuật toán Hierholzer: Khởi hành từ đỉnh gốc, di chuyển tự do qua các cạnh chưa duyệt cho đến khi quay về đỉnh xuất phát tạo thành một chu trình đơn, xóa các cạnh đã duyệt này khỏi đồ thị và đẩy lộ trình vào một Ngăn xếp (Stack). Nếu trên đường đi có các đỉnh vẫn còn cạnh chưa duyệt, ta tìm một chu trình con từ đỉnh đó và lồng ghép ngược lại vào đường đi ban đầu thông qua thao tác pop Stack. Độ phức tạp thời gian cực kỳ tối ưu: $O(E)$ vì mỗi cạnh chỉ bị duyệt đúng một lần.

3. BẢNG PHÂN CHIA CÔNG VIỆC NHÓM 5 NGƯỜI

Thành viên	Nhóm Thuật toán phụ trách	Chi tiết phân việc (Thuật toán + GUI)
Thành viên 1	Duyệt đồ thị & Biểu diễn đồ thị	- Viết chuyển đổi biểu diễn đồ thị (Ma trận kề, ds kề, ds cạnh cho cả đồ thị vô hướng & có hướng) ở utils/conversions.py. - Cài đặt thuật toán duyệt BFS & DFS thuần và generator. - Tích hợp hiển thị biểu diễn đồ thị và hoạt họa BFS/DFS lên GUI.
Thành viên 2	Đường đi ngắn nhất & Đồ thị hai phía	- Cài đặt giải thuật Dijkstra sử dụng Priority Queue (Min-Heap), kiểm soát trọng số âm. - Cài đặt thuật toán kiểm tra Đồ thị 2 phía (tô màu BFS/DFS), truy vết chu trình lẻ gây xung đột. - Tích hợp highlight lộ trình Dijkstra và tô màu hai phân hoạch lên GUI.
Thành viên 3	Cây khung nhỏ nhất (MST)	- Cài đặt thuật toán Prim (dùng Heap). - Cài đặt thuật toán Kruskal sắp xếp cạnh kết hợp cấu trúc dữ liệu DSU (Disjoint Set Union) để kiểm tra chu trình và hợp nhất nhanh. - Tích hợp trực quan hóa cây khung MST lên GUI.
Thành viên 4	Luồng cực đại (Max Flow)	- Cài đặt thuật toán luồng cực đại Ford-Fulkerson (Edmonds-Karp) dùng BFS tìm đường tăng luồng ngắn nhất trên đồ thị dư. - Tích hợp hiển thị dòng luồng flow/capacity trên các cạnh và highlight các đường tăng luồng qua từng bước. - Phụ giúp Thành viên 1 thiết kế khung Canvas chính.
Thành viên 5	Đường đi & Chu trình Euler	- Cài đặt giải thuật Fleury (duyet kiểm tra và tránh các cạnh cầu - bridge). - Cài đặt giải thuật Hierholzer tối ưu $O(E)$ sử dụng cơ chế lưu trữ Stack. - Tích hợp mô phỏng bước đi của Fleury và cơ chế push/pop đỉnh vào Stack của Hierholzer lên GUI.

4. CHI TIẾT CODE PHẢI TRIỂN KHAI CHO TỪNG THÀNH VIÊN

4.1. Thành viên 1: Mô-đun Conversions và BFS/DFS

Mã khung lẫn trình (Skeleton Code):

```
# Tập tin: utils/conversions.py
def to_adjacency_matrix(nodes, edges, is_directed=False):
    n = len(nodes)
    matrix = [[0] * n for _ in range(n)]
    for u, v, w in edges:
        matrix[u][v] = w
        if not is_directed:
            matrix[v][u] = w
    return matrix

# Tập tin: algorithms/pure/traversal.py
from collections import deque
def bfs(graph, start):
    visited, visited_set = [], {start}
    queue = deque([start])
    edges = []
    while queue:
        u = queue.popleft()
        visited.append(u)
        for v, w in sorted(graph.get(u, []), key=lambda x: x[0]):
            if v not in visited_set:
                visited_set.add(v)
                queue.append(v)
                edges.append((u, v, w))
    return visited, edges
```

4.2. Thành viên 2: Thuật toán Dijkstra và Đồ thị hai phía

Mã khung lần trình (Skeleton Code):

```
# Tập tin: algorithms/pure/dijkstra.py
import heapq
def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    prev = {node: None for node in graph}
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        current_dist, u = heapq.heappop(pq)
        if current_dist > dist[u]: continue
        for v, weight in graph.get(u, []):
            if weight < 0: raise ValueError("Trọng số âm không hỗ trợ!")
            if current_dist + weight < dist[v]:
                dist[v] = current_dist + weight
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, prev

# Tập tin: algorithms/pure/bipartite.py
def check_bipartite(graph):
    color = {} # Lưu đỉnh và màu của nó (0 hoặc 1)
    parent = {}
    for start_node in graph:
        if start_node not in color:
            stack = [(start_node, 0)]
            color[start_node] = 0
            while stack:
                u, c = stack[-1]
                unvisited_found = False
                for v, _ in graph[u]:
                    if v not in color:
                        color[v] = 1 - c
                        parent[v] = u
                        stack.append((v, 1 - c))
                        unvisited_found = True; break
                    elif color[v] == color[u]:
                        # Phát hiện chu trình lẻ
                        cycle = [v, u]
                        curr = u
                        while curr in parent and curr != v:
                            curr = parent[curr]
                        cycle.append(curr)
                        return False, cycle[::-1]
            if not unvisited_found: stack.pop()
    return True, color
```

4.3. Thành viên 3: Cây khung nhỏ nhất MST (Prim và Kruskal)

Mã khung lần trình (Skeleton Code):

```
# Tập tin: algorithms/pure/kruskal.py
def kruskal(node_count, edges):
    parent = list(range(node_count))
    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x]) # Path compression
        return parent[x]
    def union(x, y):
        root_x, root_y = find(x), find(y)
        if root_x != root_y:
            parent[root_x] = root_y
            return True
        return False
    sorted_edges = sorted(edges, key=lambda e: e[2])
    mst = []
    total = 0
    for u, v, w in sorted_edges:
        if union(u, v):
            mst.append((u, v, w))
            total += w
            if len(mst) == node_count - 1:
                break
    return mst, total

# Tập tin: algorithms/pure/prim.py
import heapq
def prim(node_count, graph, start=0):
    visited = {start}
    mst = []
    total = 0
    pq = []
    for v, w in graph.get(start, []):
        heapq.heappush(pq, (w, start, v))
    while pq and len(visited) < node_count:
        w, u, v = heapq.heappop(pq)
        if v not in visited:
            visited.add(v)
            mst.append((u, v, w))
            total += w
            for next_v, next_w in graph.get(v, []):
                if next_v not in visited:
                    heapq.heappush(pq, (next_w, v, next_v))
    return mst, total
```

4.4. Thành viên 4: Luồng cực đại Ford-Fulkerson (Edmonds-Karp)

Mã khung lần trình (Skeleton Code):

```
# Tập tin: algorithms/pure/max_flow.py
from collections import deque

def bfs_augmenting_path(graph, capacity, flow, s, t, parent):
    visited = {s}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in graph[u]:
            residual = capacity[(u, v)] - flow.get((u, v), 0)
            if v not in visited and residual > 0:
                visited.add(v)
                parent[v] = u
                if v == t: return True
                queue.append(v)
    return False

def edmonds_karp(node_count, edges, s, t, is_directed=False):
    graph = {i: set() for i in range(node_count)}
    capacity = {}
    for u, v, w in edges:
        graph[u].add(v)
        capacity[(u, v)] = capacity.get((u, v), 0) + w
        if not is_directed:
            graph[v].add(u)
            capacity[(v, u)] = capacity.get((v, u), 0) + w
    flow = {}
    max_flow_val = 0
    parent = {}
    while bfs_augmenting_path(graph, capacity, flow, s, t, parent):
        path_flow = float('inf')
        curr = t
        while curr != s:
            p = parent[curr]
            residual = capacity[(p, curr)] - flow.get((p, curr), 0)
            path_flow = min(path_flow, residual)
            curr = p
        curr = t
        while curr != s:
            p = parent[curr]
            flow[(p, curr)] = flow.get((p, curr), 0) + path_flow
            flow[(curr, p)] = flow.get((curr, p), 0) - path_flow
            curr = p
        max_flow_val += path_flow
        parent = {}
    return max_flow_val, flow
```

4.5. Thành viên 5: Đường đi & Chu trình Euler (Fleury và Hierholzer)

Mã khung lẫn trình (Skeleton Code):

```
# Tập tin: algorithms/pure/hierholzer.py
def find_eulerian_path_or_circuit_hierholzer(graph):
    adj = {u: {} for u in graph}
    for u in graph:
        for v, w in graph[u]:
            adj[u][v] = adj[u].get(v, 0) + 1
    # Tìm đỉnh bậc lẻ để chọn điểm bắt đầu (nếu là đường đi Euler)
    start_node = next(iter(graph))
    degrees = {u: sum(adj[u].values()) for u in graph}
    odd_nodes = [u for u in degrees if degrees[u] % 2 != 0]
    if len(odd_nodes) == 2:
        start_node = odd_nodes[0]
    stack = [start_node]
    path = []
    while stack:
        u = stack[-1]
        neighbors = [v for v in adj[u] if adj[u][v] > 0]
        if neighbors:
            v = neighbors[0]
            adj[u][v] -= 1
            adj[v][u] -= 1
            stack.append(v)
        else:
            path.append(stack.pop())
    return path[::-1]

# Tập tin: algorithms/pure/fleury.py
# (Để viết Fleury, Thành viên 5 cần định nghĩa thêm hàm kiểm tra cạnh cầu - bridge)
def is_bridge(u, v, graph):
    # Kiểm tra xem việc xóa cạnh (u, v) có làm mất tính liên thông hay không
    # Bằng cách chạy đếm số thành phần liên thông bằng DFS/BFS
    pass
```

5. QUY TRÌNH PHỐI HỢP & TIẾN ĐỘ THỰC HIỆN

Nhóm tuân thủ mô hình phát triển tích hợp liên tục (CI) theo 4 bước sau:

1. Viết giải thuật thuần trước: Viết toàn bộ code trong thư mục pure/ và chạy kiểm thử CLI. Đảm bảo thuật toán độc lập hoàn toàn với Tkinter.
2. Chuyển sang generator: Copy thuật toán ra thư mục ngoài, chèn các từ khóa yield sau mỗi chu trình lặp chính để gửi thông tin về GUI.
3. Liên kết giao diện: Tự viết hàm vẽ kết quả trung gian nhận được từ Generator của mình lên Canvas chính của chương trình trong file gui.py.
4. Tạo đồ thị mẫu và test chéo: Tự thiết kế ít nhất 1 đồ thị mẫu đặc trưng cho thuật toán của mình dưới dạng JSON để người dùng dễ kiểm thử nhanh.

6. TIÊU CHUẨN HOÀN THÀNH & CLEAN CODE (DoD)

Một phần việc được coi là hoàn thành 100% khi đáp ứng đầy đủ các tiêu chuẩn học thuật và lập trình sau:

- Chính xác tuyệt đối (Deterministic): Các thuật toán duyệt hoặc chọn cạnh kề phải luôn thực hiện sắp xếp danh sách đỉnh kề theo nhãn tăng dần trước khi duyệt để khớp với kết quả giải tay lý thuyết.
- Mượt mà và Không đơ ứng dụng: Hoạt họa chạy trên GUI phải sử dụng luồng generator tuần tự kết hợp hàm after() của Tkinter, tuyệt đối không được dùng vòng lặp vô hạn hay hàm time.sleep() trong luồng chính.
- Chuẩn hóa Clean Code: Đặt tên biến và hàm theo chuẩn PEP 8 (tiếng Anh rõ ràng). Bắt buộc phải viết docstring mô tả chi tiết kiểu dữ liệu đầu vào và đầu ra cho tất cả các hàm.
- Quản lý qua Git: Từng thành viên commit code lên nhánh riêng (ví dụ: feature/prim-kruskal), test chạy thử không có lỗi mới được gửi Pull Request gộp vào nhánh chính main.